



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Multi AI Agents Workflow for Content Creation

TGS Ref No: TGS-2023036153

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 2.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	13 Sep 2026	Multi-agent content strategy courseware aligned to the current course page and TSC; GreenLeaf Café case; 12 labs.	Tertiary Infotech Academy
2.0	8 October 2026	Revamped in the house design of Agentic AI Applications with Codex (TGS-2023041081). One central case — Horizon Wealth Planning, a Singapore financial-planning firm — across 14 labs, each in its own folder: Claude Cowork sub-agents (research, ideation, personas, cadence, skills, measurement), Codex (website, compliance subagent, approval gate, video), Claude Code subagents and agent teams (social, newsletter and video teams with LinkedIn, Facebook and YouTube publishing behind a human approval gate) and Hermes Agent bots on a Kanban board. 2-hour assessment from 4:00 PM Day 2.	Dr. Alfred Ang

TABLE OF CONTENTS

How to Use This Guide.....	5
The Scenario: Horizon Wealth Planning.....	5
About This Course.....	5
Learning Outcomes.....	5
One Firm, One Content Studio of Agents.....	6
Meet Horizon Wealth Planning.....	6
The Right Tool for Each Job.....	6
Course Outline.....	7
Lab Materials.....	7
Lesson Plan (9:00 AM – 6:00 PM).....	7
Topic 1 — Multi-AI-Agent Content Ideation and Digital Storyboarding.....	8
Key ideas for Lab 1.....	8
From Prompts to Agent Teams.....	8
Why a Team of Agents?.....	8
The Agent Loop.....	8
Four Tools, One Studio.....	9
Get the Tools Ready.....	9
Claude Cowork and Sub-agents.....	9
The Research Team.....	9
Research You Can Trust.....	9
Lab 1 — A Research Team of Sub-agents.....	10
Key ideas for Lab 2.....	11
Codex: Plan Before You Build.....	11
The Retirement Calculator.....	11
Checklist and Contact Form.....	12
Lab 2 — Plan and Build the Horizon Website with Codex.....	13
Key ideas for Lab 3.....	14
Financial Content Is Regulated.....	14
A Reviewer That Only Reports.....	14
Lab 3 — A Compliance Subagent, Then Publish.....	15
Key ideas for Lab 4.....	16
From Objective to Storyboard.....	16
The Five-Beat Storyboard.....	16
Lab 4 — Ideation Sub-agents and a Digital Storyboard.....	16
Topic 1 recap.....	18
Where You Are Now.....	18
Topic 2 — Audience Research and Content Requirement Analysis.....	18
Key ideas for Lab 5.....	18
Evidence Before Personas.....	18
Observed, Inferred or Low Confidence?.....	18
Data Agents Must Count.....	18
Lab 5 — Audience Research and Personas from Evidence.....	19
Key ideas for Lab 6.....	20
Finding the Right Frequency.....	20
Lab 6 — Content Cadence and the Editorial Calendar.....	20
Key ideas for Lab 7.....	21

Skills vs Project Rules.....	21
One Skill Set for Every Tool.....	22
The Four Studio Skills.....	22
Lab 7 — Studio Skills: Voice, Facts, Compliance and Formats.....	22
Topic 2 recap.....	23
Day 1: Research, Build and Plan.....	23
Topic 3 — Multi-Channel Content Creation and Agent Workflow Coordination.....	23
Key ideas for Lab 8.....	24
Subagents vs Agent Teams.....	24
Which Pattern When?.....	24
Anatomy of a Subagent.....	24
Running an Agent Team.....	25
Human in the Loop, Enforced.....	25
The Hook That Holds the Gate.....	25
The Publishing APIs.....	26
Lab 8 — The Human Approval Gate and the Publishers.....	26
Key ideas for Lab 9.....	27
The Social Media Team.....	27
Lab 9 — The Social Media Agent Team: LinkedIn and Facebook.....	28
Key ideas for Lab 10.....	29
The Newsletter Team.....	29
Lab 10 — The Newsletter Agent Team: the Sunny Sunday Email.....	29
Key ideas for Lab 11.....	31
The Video Team.....	31
Lab 11 — The Video Agent Team: Script to YouTube.....	31
Topic 3 recap.....	32
Where You Are Now.....	32
Topic 4 — Content Distribution, Strategy Guidelines and Responsible AI Practices.....	32
Key ideas for Lab 12.....	33
What Is Hermes Agent?.....	33
The Hermes Kanban Board.....	33
A Role Pipeline on the Board.....	33
Claude Agent Teams or Hermes Kanban?.....	34
Lab 12 — Hermes Agent Bots on a Kanban Board.....	34
Key ideas for Lab 13.....	35
Always-On Delivery.....	35
Lab 13 — Always On: a Telegram Bot That Asks Before It Acts.....	36
Key ideas for Lab 14.....	37
Responsible AI in the Studio.....	37
Measure What Matters.....	37
Lab 14 — Measure, Govern and Write the Playbook.....	37
Course Summary.....	38
How Each Pattern Is Triggered.....	38
Horizon: Research to an Always-On Studio.....	39
Quick Command Reference.....	39
Support.....	40
Assessment flow.....	40

How to Use This Guide

This guide carries the full step-by-step for every lab in the course. The slides explain the idea; this guide is what you follow at the keyboard. Every lab also has its own folder in the lab pack with a README, the prompts as Markdown and PDF, the assets you need and an evidence checklist.

Before you start, have ready:

- The Claude desktop app on a Pro plan or above (Claude Cowork and Claude Code), and the Claude Code CLI for agent teams.
- The ChatGPT desktop app on a plan that includes Codex.
- Node.js 22, Python 3 with Pillow, ffmpeg and Git; a GitHub account.
- A personal Google account for Gmail — never an employer's.
- The lab pack, unzipped. Keep one working folder, horizon-studio, from Lab 1 to Lab 14.

Prompts appear as shaded quotes — paste them as written. Code, commands and configuration appear in grey monospace blocks. Product menus change between releases: if a name here differs from your screen, follow the screen and tell the trainer.

The Scenario: Horizon Wealth Planning

Rachel Goh founded Horizon Wealth Planning in 2010. Fifteen years and 1,200 clients later, almost every new client still comes through a referral — and referrals are slowing. Younger Singaporeans look for money help on LinkedIn, Facebook, YouTube and in their inbox, and Horizon is invisible there.

In October 2026 Rachel sets a goal: 40 booked free chats a month by March 2027, without hiring a marketing team. She has three planners, a part-time marketing executive (Jun Wei), S\$4,000 a month — and AI agents. You are her content studio for the next two days:

- Research — who to reach, the numbers, the competition (Cowork sub-agents)
- Build — the website every post points to (Codex)
- Plan — ideas, storyboard, personas, cadence, skills (Cowork)
- Create — social, newsletter and video teams (Claude agent teams)
- Run — always-on agent bots, measured and governed (Hermes, Cowork)

Every lab starts from what the previous lab produced. Financial content is regulated, so a person approves every piece before it goes out. The data is fictitious but internally consistent, and every email address resolves to your own inbox.

About This Course

Learning Outcomes

By the end of the two days you will be able to:

- **LO1 · Conceptualise** — Conceptualise content ideas that meet marketing objectives, and map them into digital storyboards.
- **LO2 · Identify** — Identify content requirements from customer preferences, and decide how often to publish.

- **LO3 · Determine** — Determine content types and styles, and the modes and processes for distributing content.
- **LO4 · Develop** — Develop guidelines for executing the content strategy, with suitable delivery modes and responsible AI practices.

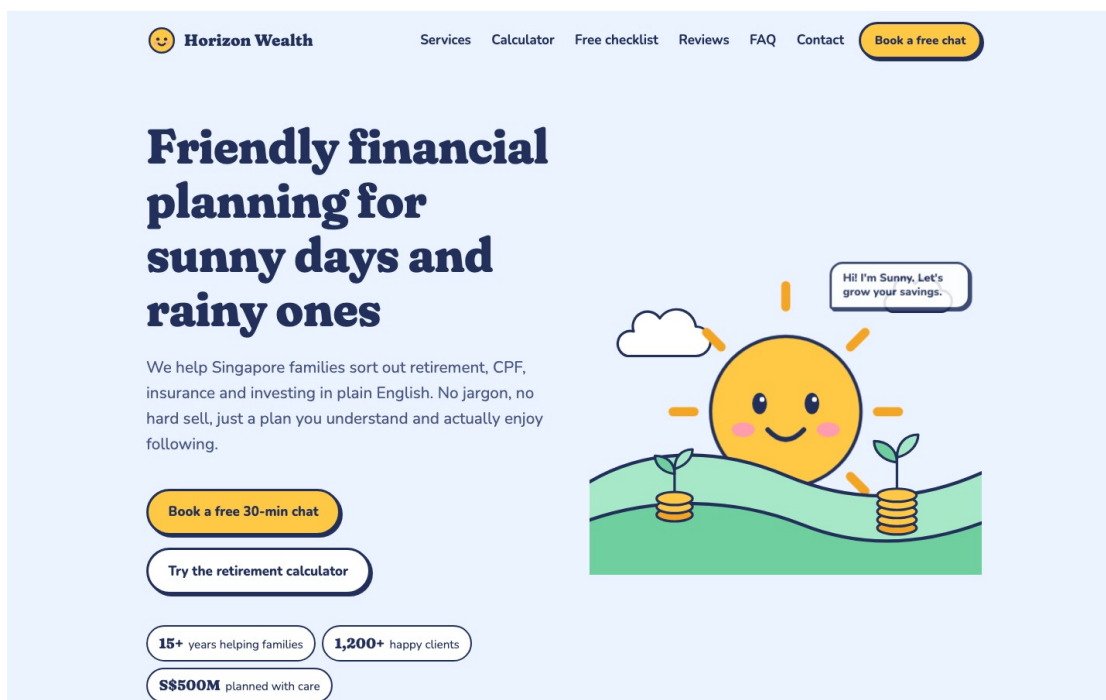
One Firm, One Content Studio of Agents

Horizon Wealth Planning — independent financial planners in Singapore since 2010. Goal: 40 booked free chats a month by March 2027.

1. **Research** — A team of Cowork sub-agents: market, numbers, competitors, recommendation.
2. **Build** — Codex builds and publishes the website every post points to.
3. **Plan** — Ideas, storyboard, personas, cadence and shared skills.
4. **Create** — Claude agent teams write social posts, the newsletter and a video.
5. **Run** — Hermes agent bots on a Kanban board, measured and governed.

Meet Horizon Wealth Planning

The firm's website — the reference build for Lab 2. Every post, email and video in this course points back here.



- **Plain English** — Retirement, CPF, insurance and investing — no jargon, no hard sell.
- **One goal** — Turn visitors into booked free 30-minute chats.
- **Regulated** — Financial content: general information only, never a promised return.

The Right Tool for Each Job

Four tools, one studio. Pick by the job, not by habit.

Tool	Use it for	In this course
Claude Cowork	Business work on your files and apps: research, analysis, skills, email drafts	Labs 1, 4–7, 10, 14

Codex	Building software: the website, scripts, the approval gate, the video	Labs 2–3, 8, 11
Claude Code	Subagent definitions and agent teams that write and review together	Labs 9–11
Hermes Agent	Open-source agent bots on any model, a Kanban board, schedules and chat apps	Labs 12–13

Tip: Every external post, email and video passes one gate: a person approves the exact version.

Course Outline

Four topics over two days. Every topic ends in labs that move Horizon forward.

1. **T1 · Ideation and Storyboarding** — Research sub-agents, the website, a compliance subagent, the storyboard. Labs 1–4.
2. **T2 · Audience and Requirements** — Personas from evidence, cadence and calendar, studio skills. Labs 5–7.
3. **T3 · Creation and Coordination** — Approval gate; social, newsletter and video agent teams. Labs 8–11.
4. **T4 · Distribution and Responsible AI** — Hermes bots and Kanban, always-on delivery, measure and govern. Labs 12–14.

Lab Materials

Fourteen labs, each in its own folder with a README, prompts (MD and PDF), assets and an evidence checklist.

Day	Labs	Tool
1	1 Research team · 2 Build the site · 3 Compliance + publish	Cowork, Codex
1	4 Ideas + storyboard · 5 Personas · 6 Cadence · 7 Skills	Cowork
2	8 Approval gate · 9 Social team · 10 Newsletter team	Codex, Claude Code, Cowork
2	11 Video team · 12 Hermes Kanban · 13 Telegram bot	Claude Code, Codex, Hermes
2	14 Measure, govern, playbook	Cowork

Tip: Labs build on each other: one horizon-studio folder from Lab 1 to Lab 14.

Lesson Plan (9:00 AM – 6:00 PM)

Tea breaks 10 min, lunch 45 min. Full timings and slide numbers are in the Lesson Plan.

Block	Day 1 — Research, Build, Plan	Day 2 — Create, Distribute, Govern
Morning 1	9:00–10:45 Welcome · Topic 1 · Lab 1	9:00–10:45 Topic 3 · Labs 8–9
Morning 2	10:55–12:40 Labs 2–3	10:55–12:30 Labs 10–11 · recap
Afternoon 1	13:25–15:25 Lab 4 · Topic 2 · Lab 5	13:15–14:50 Topic 4 · Labs 12–13
Afternoon 2	15:35–18:00 Labs 6–7 · review	15:00–16:00 Lab 14 · summary · 16:00–18:00 Assessment

Topic 1 — Multi-AI-Agent Content Ideation and Digital Storyboarding

Slides 18–53. In this topic you will:

- From one chatbot to teams of agents
- Claude Cowork, Codex, Claude Code and Hermes — and the agent loop
- Research with sub-agents; build and check the website
- Ideas from parallel agents, then a five-beat storyboard

Key ideas for Lab 1

From Prompts to Agent Teams

- **2023 · Prompt Engineering** — ChatGPT goes mainstream. Craft the wording of one prompt.
- **2024 · Tools and MCP** — Models call tools. Anthropic open-sources the Model Context Protocol (Nov 2024).
- **2024 · Context Engineering** — Fill the context with exactly what the next step needs: files, memory, retrieval, skills.
- **2025 · Harness Engineering** — Agents ship inside a harness — Claude Code, Codex. The loop, tools and checks around the model.
- **2026 · Agent Teams** — Many agents with their own roles: Claude Code agent teams, Codex subagents, Hermes Agent and its Kanban.

Years show when each practice took hold; the names came later. One agent is a helper. A team with a reviewer and a human gate is a studio.

Why a Team of Agents?

Ask for sub-agents and the main agent runs specialists in parallel, each in its own context, then combines what they return.

- **Faster** — Independent research runs at the same time, not one after another.
- **Focused** — Each agent holds only its own job — no context full of other people's work.
- **Specialised** — Its own role, tools, model and rules.
- **Second opinion** — A reviewer that did not write the post checks it.
- **The cost** — More tokens and more to review. Split only work that is truly independent.

The Agent Loop

Every tool in this course runs the same loop. Sub-agents and teams are more loops, running side by side.

1. **Context** — Reads your prompt, the project rules, files and skills.
2. **Plan** — Decides the next step — or proposes a plan for you to approve.
3. **Act** — Calls a tool: searches the web, writes a file, runs a script, starts a sub-agent.
4. **Verify** — Checks the result: a test, a count, a reviewer.
5. **Repeat** — Until the goal is met — then reports what changed and how it was checked.

Four Tools, One Studio

All four run agents. They differ in what they are built for.

- **Claude Cwork** — In the Claude desktop app. Works on a folder and your connected apps; builds Live Artefacts; runs sub-agents in parallel.
- **Codex** — In the ChatGPT desktop app. Builds software in a project: /plan, AGENTS.md, subagents, hooks.
- **Claude Code** — Terminal, IDE or desktop. Subagent files in .claude/agents/ and agent teams with a shared task list.
- **Hermes Agent** — Open source (MIT) from Nous Research. Any model; profiles as bots; a Kanban board; cron; Telegram.

Get the Tools Ready

Install once; the labs tell you which tool to open.

- **Claude desktop** — Cwork (and Claude Code) — Pro plan or above
 - **ChatGPT desktop** — Codex — chatgpt.com/download
1. **Claude desktop** — Install from claude.com/download and sign in on a paid plan. Cwork is in the sidebar.
 2. **Claude Code CLI** — `curl -fsSL https://claude.ai/install.sh | bash` (Windows: `irm https://claude.ai/install.ps1 | iex`).
 3. **ChatGPT desktop** — For Codex. Sign in on a plan that includes Codex.
 4. **Hermes Agent** — Lab 12 — installed in class with the trainer's API key.
 5. **Node.js 22, Python 3, Git** — For the scripts, the video and GitHub Pages.

Note: Claude Code needs a Pro, Max, Team, Enterprise or Console account.

Claude Cwork and Sub-agents

Cwork brings Claude's agent loop to everyday business work.

- **Point it at a folder** — Cwork reads and writes the files in horizon-studio, and asks before it changes anything important.
- **Ask for sub-agents** — Say "use sub-agents, one per role, in parallel". Each works in its own context; only results come back.
- **Get a file, not a chat** — Reports, CSVs, Live Artefacts and email drafts — saved where the next lab can use them.

The Research Team

Lab 1: four specialists, three in parallel, one to decide what it means.

1. **Market researcher** — Who looks for planning, on which channels, why now. Web, cited.
2. **Business analyst** — The funnel: visits → enquiries → chats → plans. Counts.
3. **Competitive analyst** — Six named competitors: offer, pricing model, content.
4. **Strategy advisor** — Reads all three; writes the recommendation.
5. **You** — Read it, challenge it, change it. It is your call.

Research You Can Trust

Agents write plausible numbers fluently. Make them prove each one.

- **Cite or say UNKNOWN** — Every figure needs a source and a date. A gap marked UNKNOWN beats a confident guess.
- **Keep internal data internal** — services.csv has INTERNAL fee columns for analysis. They never appear in anything public.
- **Read the sources** — Open two of the links yourself. If a source does not say it, the report does not either.

Lab 1 — A Research Team of Sub-agents

The story so far: October 2026. Rachel wants 40 booked chats a month by March. Before she spends a dollar, her partners want three answers: who to reach, what the funnel needs, and what competitors already publish.

Goal: Before Horizon spends a dollar on content, find out who to reach, what the numbers need and what competitors do — with four specialist agents instead of one overloaded chat.

You'll build: research/: market research, business analysis, competitive analysis and a recommendation, every figure cited

Surface: Claude Cowork · **Time:** 45 min · **Slides:** 27–31

Lab folder: labs/lab-01-research-team-of-subagents/ — assets: business-brief.md, firm-metrics.csv, services.csv, research-agents-spec.md

Step-by-step

1. **Make the studio folder** — Create horizon-studio with a data/ folder. Copy this lab's assets into data/.
2. **Open Cowork** — In the Claude desktop app choose Cowork, then point it at the horizon-studio folder.
3. **Brief the team** — Paste Prompt A. Cowork confirms the four roles and the plan — correct anything before it starts.
4. **Run them in parallel** — Paste Prompt B. Watch three sub-agents work at once in the progress panel.
5. **Audit the sources** — Open competitive-analysis.md. Every figure needs a source and a date; gaps say UNKNOWN.
6. **Decide** — Read recommendation.md. Change one thing you disagree with — it is your recommendation now.

PROMPT A — Cowork: brief the team

I'm the marketing lead at Horizon Wealth Planning, an independent financial-planning firm in Singapore. Read data/business-brief.md and data/research-agents-spec.md.

We will run a research team of four sub-agents:

1. market-researcher — Singapore demand for financial planning: who, which channels, why now.
2. business-analyst — our funnel from data/firm-metrics.csv: what 40 chats a month needs.
3. competitive-analyst — six named competitors:
offer, pricing model, channels, content themes.
4. strategy-advisor — reads the other three and writes the recommendation.

Rules for all: cite every figure with source and date, or write UNKNOWN. Never publish the INTERNAL columns. Confirm each role in one line and show me your plan. Do not start yet.

PROMPT B — Cowork: run the team

Run the research team now, using sub-agents.

- Start market-researcher, business-analyst and competitive-analyst in parallel, each as its own sub-agent, each saving its report to research/.
- When all three are done, the strategy-advisor writes research/recommendation.md exactly as research-agents-spec.md describes. Then give me a five-line summary and a list of everything marked UNKNOWN.

Check your work

- ☐ Cowork confirmed all four roles before starting.
- ☐ Three research sub-agents ran at the same time.
- ☐ research/ holds market-research, business-analysis, competitive-analysis and recommendation.
- ☐ The competitive analysis names six real competitors, each with a source and a date.
- ☐ The business analysis shows conversion rates and the visits needed for 40 chats a month.
- ☐ No uncited figure; no INTERNAL fee anywhere in the reports.

If it goes wrong

- **Cowork did it all itself** — Say it directly: "use sub-agents — one per role, in parallel".
- **No time for research** — Use recommendation-sample.md from Lab 2's assets and carry on.

Stretch

- Add a fifth sub-agent, regulatory-scout, that lists the MAS and PDPA rules a financial firm's marketing must follow — with sources.

Why it matters: A sub-agent does its research in its own context and hands back only the result. Your main conversation stays clear for the decision.

Key ideas for Lab 2

Codex: Plan Before You Build

Create a local project, add the horizon-studio folder, then plan.

- **/plan first** — Codex reads the brief and asks what it cannot determine. Every question is a guess it was about to make.
- **Build from the brief** — site-brief.md, brand.md and the CSVs — not the model's idea of a financial website.
- **AGENTS.md** — /init drafts the project rules. Trim to What this is, Commands, Conventions, Boundaries.

The Retirement Calculator

The acceptance test: age 30, retire 62, S\$20,000, S\$800 a month, 4% → S\$693,138.


Horizon Wealth

[Services](#)
[Calculator](#)
[Free checklist](#)
[Reviews](#)
[FAQ](#)
[Contact](#)
[Book a free chat](#)

How big could your nest egg grow?

Play with the numbers to see a rough picture of your savings at retirement. It takes 30 seconds, and nothing you type leaves your browser.

Your age now

Age you'd like to retire

Savings so far (not CPF)

Saved each month

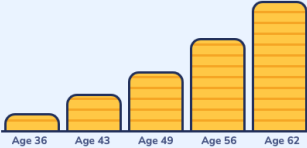
Average yearly return (not guaranteed)

4%

At 62 you could have about

\$S\$693,138

That's roughly S\$2,310 a month over a 25-year retirement, on top of your CPF LIFE payouts.



Age 36 Age 43 Age 49 Age 56 Age 62

[Ask a planner to check my numbers](#)

Illustration only. Assumes a steady return and fixed monthly savings, before inflation and fees. Returns are not guaranteed and this is not financial advice.

- **Monthly compounding** — $\text{Rate} \div 12, \text{years} \times 12$. The test number proves the maths.
- **Labelled** — "Illustration only ... not guaranteed ... not financial advice." Word for word.
- **Private** — Nothing typed leaves the browser.

Checklist and Contact Form

The two places a visitor becomes a lead.

Free: the 10-minute Money Check-up Checklist

Ten quick questions every Singapore household should be able to answer. Tick them off and you'll know exactly where you stand, and what to fix first.

- ☒ Is your emergency fund big enough?
- ☒ Are your CPF nominations and will up to date?
- ☒ Do you have gaps (or overlaps) in your insurance?
- ☒ Are you using SRS to cut your income tax?
- ☒ Do you know your retirement number?

Instant access, plus one friendly money tip a month. No spam, unsubscribe any time.

Get your free checklist

Pop in your email and the checklist unlocks right here, ready to tick off or print.

[Unlock my free checklist](#)

- **A lead magnet** — The 10-item Money Check-up, unlocked with an email.
- **Consent** — The contact form will not submit without the PDPA consent box.
- **No backend** — A thank-you message in class; a form service in production.

Lab 2 — Plan and Build the Horizon Website with Codex

The story so far: The research is back: mid-career families and pre-retirees, on LinkedIn, Facebook and email. Every post will point to the website — and the current one is out of date. Rachel wants a new site this week, built from the brief so nothing is invented.

Goal: Every post, email and video will point somewhere. Build the website first — planned before a single file changes, and driven by the brief, so nothing is invented.

You'll build: docs/index.html: hero, services, retirement calculator, checklist, reviews, FAQ, contact form with PDPA consent

Surface: Codex · **Time:** 45 min · **Slides:** 35–38

Lab folder: labs/lab-02-build-the-website-with-codex/ — assets: site-brief.md, brand.md, faq.md, testimonials.csv, checklist-items.md, recommendation-sample.md

Step-by-step

1. **Add the brief** — Copy the assets into horizon-studio/data/. No Lab 1 recommendation? Copy recommendation-sample.md to research/recommendation.md.
2. **Open it in Codex** — In the ChatGPT desktop app open Codex, create a local project and add the horizon-studio folder. Run git init if it is not a repo yet.
3. **Plan first** — Type /plan, then paste the prompt. Read every question Codex asks.
4. **Answer and narrow** — Answer the questions. Cut one step you did not ask for, then approve.
5. **Serve it** — Run `python3 -m http.server 8080 --directory docs` and open `http://localhost:8080`.
6. **Test it** — Calculator: 30, 62, S\$20,000, S\$800, 4% must show S\$693,138. Try it at 375px wide.
7. **Write the rules** — Run /init. Trim AGENTS.md to What this is, Commands, Conventions, Boundaries — under 60 lines.

PROMPT — Codex, after /plan

Build the Horizon Wealth Planning website in docs/index.html (GitHub Pages will serve docs/). Read data/site-brief.md, data/brand.md, data/services.csv, data/faq.md, data/testimonials.csv, data/checklist-items.md and research/recommendation.md first.

MUST HAVE

- Every section in site-brief.md, in that order.
- Six service cards — never the INTERNAL columns.
- A retirement calculator with monthly compounding and the illustration note word for word.
- The checklist unlocked by email; contact form with a REQUIRED PDPA consent box; no backend.
- The footer disclaimer word for word.

CONSTRAINTS

- One HTML file, inline CSS and JS, no framework.
- Accessible: labels, alt text, focus, contrast.

DONE WHEN

- The calculator test gives S\$693,138.
- No horizontal scroll at 375px.

Check your work

- ☐ Plan mode showed steps AND questions before any edit.
- ☐ You answered its questions instead of letting it guess.
- ☐ The calculator test shows S\$693,138 with its illustration note.
- ☐ Six service cards; no INTERNAL fee or hours anywhere.
- ☐ The contact form will not submit without consent.
- ☐ No horizontal scroll at 375px; AGENTS.md is under 60 lines.

If it goes wrong

- **The calculator is off** — Ask Codex to use monthly compounding: $r = \text{rate}/12$, $n = \text{years} \times 12$.
- **The page is blank** — Serve it over `http://` — opening the file directly can block scripts.

Stretch

- Add Open Graph tags so a shared link shows a card with the tagline and Sunny.

Why it matters: Build from the brief, not from memory. The model knows what a financial website looks like; only the brief knows what Horizon is allowed to say.

Key ideas for Lab 3

Financial Content Is Regulated

Twelve rules every Horizon page, post, email and video must pass (compliance-checklist.md).

Rule	What it means	If broken
C1 · Not advice	General information only — say so	Critical
C2 · No promises	No guaranteed or "risk-free" returns	Critical
C3 · Label illustrations	Assumptions shown; not guaranteed	Critical
C5 · No performance	Never client returns or "average" returns	Critical
C9 · Consent	Marketing only to people who opted in	Critical
C11 · Facts sheet	Every figure from facts-2026.md	Critical

Tip: MAS Guidelines on Standards of Conduct for Digital Advertising Activities apply from 25 March 2026.

A Reviewer That Only Reports

Lab 3: Codex starts a compliance reviewer as a subagent.

The writer (main agent)

- Built the page and wants it to pass
- Can edit every file
- Fixes what the reviewer reports
- Re-runs the reviewer until PASS

The reviewer (subagent)

- Did not write the page
- Reads only — never edits
- Table: words, rule, severity, fix
- PASS only with no Critical or High issue

Lab 3 — A Compliance Subagent, Then Publish

The story so far: The site looks great. Then Rachel's partner reads the 'How it works' intro she drafted — '8% a year ... guaranteed' — and goes pale. Before the site goes live, a reviewer that did not write it must check it.

Goal: Financial content is regulated. A second agent that did not write the page — and is not allowed to edit it — checks it before anyone else sees it. Then the site goes live.

You'll build: review/site-compliance.md (FAIL, then PASS) and the site live on GitHub Pages

Surface: Codex → GitHub Pages · **Time:** 35 min · **Slides:** 41–45

Lab folder: labs/lab-03-compliance-subagent-and-publish/ — assets: compliance-checklist.md, facts-2026.md, publish-checklist.md

Step-by-step

1. **Add the rules** — Copy compliance-checklist.md and facts-2026.md into data/.
2. **Run the reviewer** — Paste Prompt A. Codex starts the reviewer as a subagent — watch it in the Subagents panel.
3. **Read the report** — It must flag Rachel's "8% a year ... guaranteed" line as Critical. Note anything else it found.
4. **Fix and re-check** — Paste Prompt B. The main agent fixes; the reviewer checks again until PASS.
5. **Publish** — Create the horizon-studio repo on GitHub, then Settings → Pages → Deploy from a branch → main, /docs.
6. **Test on a phone** — Open the Pages URL on your phone and try the calculator and the contact form.

PROMPT A — Codex: the reviewer

Use a subagent as our compliance reviewer. Its job: check docs/index.html against data/compliance-checklist.md and data/facts-2026.md. It reports; it never edits a file.

Output a table — where, the exact words, the rule (C1-C12), severity (Critical / High / Low), the fix — and end with PASS or FAIL. FAIL if any Critical or High issue remains. Save the report to review/site-compliance.md.

PROMPT B — Codex: fix, re-check, ship

Fix every Critical and High issue in review/site-compliance.md, then run the compliance reviewer subagent again. Repeat until it says PASS.

Then commit and push to GitHub. Before committing, list every file you will add and confirm there is no .env, key or token among them. Give me the Pages URL:

<https://<your-user>.github.io/horizon-studio/>

Check your work

- ☐ The reviewer ran as a subagent and edited nothing.
- ☐ The first report flagged the "8% ... guaranteed" line as Critical (C2, C5).
- ☐ The final report says PASS.
- ☐ The footer disclaimer and calculator note are word for word.
- ☐ Codex listed the files before committing; no secret among them.
- ☐ The Pages URL works on your phone.

If it goes wrong

- **Pages shows 404** — Source must be Deploy from a branch, main, /docs — and the first deploy takes a minute or two.
- **The reviewer edited the page** — Say "it reports only — it must not edit any file" and run it again.

Stretch

- Save the reviewer's brief as a Codex skill with \$skill-creator so any project can use it.

Why it matters: The writer should never mark its own homework. A reviewer that cannot edit has no reason to go easy on the draft.

Key ideas for Lab 4

From Objective to Storyboard

Agents diverge; a person decides; agents make it concrete.

1. **Objective** — "Know Your Number": 40 chats a month by March.
2. **Diverge** — Three ideators in parallel, one per audience.
3. **Score** — A rubric: fit, evidence, path to a chat, effort, risk.
4. **Decide** — You choose — the human concept checkpoint.
5. **Storyboard** — Five beats, then a variant per channel.

The Five-Beat Storyboard

Every channel tells the same story with the same claims.

1. **Hook** — The viewer's own question: "Do I have enough to retire?"
2. **Tension** — What goes wrong when nobody knows their number.
3. **Proof** — One verifiable fact, e.g. the 2026 Full Retirement Sum.
4. **Resolution** — How Horizon helps — a 30-minute chat, a one-page plan.
5. **Action** — One next step: the checklist or a free chat.

Lab 4 — Ideation Sub-agents and a Digital Storyboard

The story so far: The site is live. Now the campaign: 'Know Your Number', November to January. Rachel wants ideas for three very different audiences — and a storyboard so LinkedIn, Facebook and YouTube tell the same story.

Goal: Three agents, each thinking as a different audience, give more varied ideas than one. You choose the idea; agents turn it into a storyboard every channel can follow.

You'll build: strategy/ideas.md (15 scored ideas), strategy/storyboard.md and an interactive storyboard Live Artefact

Surface: Claude Cowork (sub-agents, Live Artefact) · **Time:** 40 min · **Slides:** 48–52

Lab folder: labs/lab-04-ideation-and-storyboard/ — assets: campaign-objective.md, idea-rubric.md, storyboard-template.md

Step-by-step

1. **Add the brief** — Copy the assets into data/ and open Cowork on horizon-studio.
2. **Diverge** — Paste Prompt A — three ideator sub-agents in parallel, five ideas each.
3. **Score** — Cowork merges and scores the 15 ideas with the rubric and stops at the top five.
4. **Choose** — You pick the campaign idea — the human concept checkpoint. Say why in one line.
5. **Storyboard it** — Paste Prompt B with your choice.
6. **Check the claims** — Every beat's claim must have a source in facts-2026.md.

PROMPT A — Cowork: three ideators

Read data/campaign-objective.md, data/idea-rubric.md and research/recommendation.md.

Use three ideator sub-agents in parallel, one per audience: young professionals (25-34), mid-career families (35-49), pre-retirees (50-62). Each proposes 5 content ideas as records: title, hook, format, channel, the proof it needs, call to action.

Then merge the 15: remove duplicates, keep who proposed each, score them with the rubric and save strategy/ideas.md. Show me the top five as a table and stop — I will choose.

PROMPT B — Cowork: the storyboard

I choose idea <number>, because <one line>.

Storyboard it with data/storyboard-template.md:

five beats — hook, tension, proof, resolution, action. For each: visual, words on screen, voice-over, the claim and its source in data/facts-2026.md.

Add three channel variants that keep the same claims: LinkedIn post, Facebook post, 60-second YouTube video. Save strategy/storyboard.md and build it as a Live Artefact I can click through.

Check your work

- ☐ Three ideator sub-agents ran in parallel.
- ☐ strategy/ideas.md holds 15 ideas, each with its author and score.
- ☐ You chose the idea and recorded why.
- ☐ The storyboard has five beats; every claim has a source.
- ☐ The three variants keep the same claims.
- ☐ The storyboard Live Artefact is saved in your sidebar.

If it goes wrong

- **The ideas all sound alike** — Give each ideator a persona and a different format to favour.

Stretch

- Ask a fourth sub-agent to argue against the chosen idea, then decide whether to keep it.

Why it matters: Agents diverge; people decide. The concept checkpoint is where the campaign becomes yours.

Topic 1 recap

Where You Are Now

The research is done, the site is live and the campaign has a story. But who exactly is it for — and how often should it run?

- **Researched** · **Lab 1** — Four sub-agents, a cited recommendation you made your own.
- **Built and checked** · **Labs 2–3** — A site from the brief, a compliance subagent, live on Pages.
- **Storyboarded** · **Lab 4** — Fifteen scored ideas, your choice, five beats, three variants.

Topic 2 — Audience Research and Content Requirement Analysis

Slides 54–77. In this topic you will:

- Personas from evidence, not instinct
- Content requirements every agent can follow
- How often to publish: value, fatigue and capacity
- Skills: the studio's know-how, written once

Key ideas for Lab 5

Evidence Before Personas

Lab 5 builds personas the way an analyst would — then labels what is guesswork.

1. **Collect** — 150 survey answers, 60 enquiries, 200 checklists.
2. **Count** — With code. Every claim has an n and a file.
3. **Segment** — By age band and life stage.
4. **Persona** — Worries, channels, formats, trust — with evidence.
5. **Requirement** — What each persona needs from every piece.

Observed, Inferred or Low Confidence?

Three labels keep a persona honest.

Label	Example	Use it
Observed	30–39s chose Facebook, LinkedIn and YouTube equally (survey, n = 44)	As fact, with the count
INFERENCE	"They are short of time on weekdays" — not asked	As a hypothesis to test
LOW CONFIDENCE	The 60+ group (n = 12)	Never alone; get more data
Verbatim	"I am 54 and not sure if my CPF will be enough." (E-id)	Exactly as written, with the id

Tip: Ask "which rows show that?" of anything that sounds insightful.

Data Agents Must Count

A model that "reads" a CSV estimates. A sub-agent that runs code counts.

- **One file per sub-agent** — Survey, enquiries, checklists — three analysts in parallel, each with its own context.

- **Show the code's output** — Numbers that change between runs were estimated. Ask for the counts the code printed.
- **Requirements, with evidence** — Questions, channels, formats, length, tone, proof, CTA, never-say — each tied to a count.

Lab 5 — Audience Research and Personas from Evidence

The story so far: Jun Wei has drafted personas from instinct: 'busy young parents who love TikTok'. Rachel is not convinced. There are 150 survey answers, 60 enquiries and 200 checklists — what do they really say?

Goal: Personas made up in a meeting sound right and steer content wrong. Build them from what 150 people said, 60 enquiries and 200 checklists — with the evidence attached.

You'll build: strategy/personas.md (three personas with evidence) and strategy/content-spec.md

Surface: Claude Cowork (sub-agents) · **Time:** 40 min · **Slides:** 58–62

Lab folder: labs/lab-05-audience-research-and-personas/ — assets: survey-responses.csv, enquiries.csv, checklist-results.csv, persona-template.md, content-spec-template.md

Step-by-step

1. **Add the data** — Copy the three CSVs and two templates into data/.
2. **Analyse in parallel** — Paste Prompt A — one analyst sub-agent per data file.
3. **Build personas** — Cowork combines the findings into three personas with counts and confidence.
4. **Challenge one claim** — Pick one persona statement and ask: "which rows show that?"
5. **Write requirements** — Paste Prompt B for the content spec.

PROMPT A — Cowork: the analysts

Use three analyst sub-agents in parallel, one per file: data/survey-responses.csv, data/enquiries.csv, data/checklist-results.csv. Each one COUNTS with code — no estimating — and reports: top worries, channels, formats, trust drivers or checklist gaps, by age band, with n. Then build three personas with data/persona-template.md. Label anything the data does not show directly as INFERENCE, and any group under 20 people as LOW CONFIDENCE. Quote enquiries verbatim with their ids. Save strategy/personas.md.

PROMPT B — Cowork: requirements

From strategy/personas.md, write strategy/content-spec.md with data/content-spec-template.md: for each persona the top 3 questions to answer, channels in order, formats, length, tone, proof required, call to action and what never to say — each with the evidence (file and count) behind it.

Check your work

- ☐ Three analyst sub-agents ran, one per file.
- ☐ Every persona statement carries a count and a file.
- ☐ The 60+ group is marked LOW CONFIDENCE (n = 12).
- ☐ The most-unticked checklist item is named (retirement number).
- ☐ Enquiry quotes are verbatim, with ids.
- ☐ content-spec.md has every field for every persona.

If it goes wrong

- **The numbers change each run** — Insist the sub-agents count with code and show the code's output.

Stretch

- Ask for a Live Artefact dashboard of the survey, filterable by age band.

Why it matters: Ask "which rows show that?" of anything that sounds insightful. If the agent cannot point to rows, it is an inference — label it.

Key ideas for Lab 6

Finding the Right Frequency

Lab 6 lets twelve weeks of data set how often each channel runs.

1. **Value** — Extra clicks from one more post a week.
2. **Fatigue** — Unfollows and unsubscribes it costs.
3. **Effort** — Hours per piece, against 12 a week.
4. **Review** — Two business days for Rachel's sign-off.
5. **Calendar** — Four weeks the team can actually deliver.

Tip: A third LinkedIn post a week brings more unfollows than value. Weekly emails make unsubscribes jump.

Lab 6 — Content Cadence and the Editorial Calendar

The story so far: Jun Wei has 12 hours a week and Rachel needs two business days to review anything. Last quarter they posted when they had time. How often should each channel run — and what goes out when?

Goal: Post too little and nobody remembers you; too much and people unfollow — and the team burns out. Let the data set the frequency, then plan four weeks the team can actually deliver.

You'll build: strategy/cadence.md and strategy/calendar.csv (four weeks from 2 November 2026)

Surface: Claude Cowork · **Time:** 35 min · **Slides:** 64–68

Lab folder: labs/lab-06-cadence-and-editorial-calendar/ — assets: channel-benchmarks.csv, team-capacity.md, calendar-format.csv

Step-by-step

1. **Add the data** — Copy the assets into data/.
2. **Find the sweet spot** — Paste Prompt A. Read where an extra post stops paying off on each channel.
3. **Question it** — Ask why the newsletter is not weekly — the answer is in the unsubscribes.
4. **Plan four weeks** — Paste Prompt B.
5. **Check the buffers** — Every item's review date must be two business days before it goes out, and no holiday posts.

PROMPT A — Cowork: cadence

Using data/channel-benchmarks.csv and data/team-capacity.md, recommend how often to publish on LinkedIn, Facebook, the newsletter and YouTube. For each channel compare: the extra clicks from one more post, the unfollows or unsubscribes it costs, and the hours it takes. Show the numbers and the point where more posts stop paying off. Save strategy/cadence.md.

PROMPT B — Cowork: the calendar

Build a four-week editorial calendar from Monday 2 November 2026 in strategy/calendar.csv, in the columns of data/calendar-format.csv. Use the cadence, strategy/storyboard.md and strategy/content-spec.md.

- Every item has review_by two business days before its date; nothing on a public holiday.
- No week exceeds the team's hours. Then show me a week-by-week summary with hours.

Check your work

- ☐ cadence.md gives a frequency per channel with the evidence.
- ☐ It shows a third LinkedIn post a week costs far more unfollows than it earns.
- ☐ It explains the newsletter unsubscribe jump at weekly sends.
- ☐ calendar.csv covers four weeks in the required columns.
- ☐ Every review_by date is two business days before publishing.
- ☐ No week is over the team's 12 hours.

If it goes wrong

- **Dates fall on weekends** — Say: "publish Monday to Saturday; the newsletter is Sunday 8am only".

Stretch

- Connect Google Calendar in Cowork and add the review deadlines as events on a calendar you create for class.

Why it matters: Frequency is a capacity decision as much as an audience one. A calendar the team cannot deliver is fiction.

Key ideas for Lab 7

Skills vs Project Rules

Both are Markdown instructions. The difference is when they load.

CLAUDE.md / AGENTS.md

- Read at the start of EVERY session
- Describes the project: commands, conventions, limits
- Keep it short — it costs context every turn
- Drafted with /init

SKILL.md

- Loaded ON DEMAND, when the task matches
- One procedure, step by step
- Can be long — free until it triggers
- Created with /skill-creator

One Skill Set for Every Tool

SKILL.md is an open format. Lab 7 writes four skills once and copies the folders to each tool.

Tool	Where skills live	Labs
Claude Cowork	Customize → Skills (via /skill-creator)	Labs 7, 10, 14
Claude Code	.claude/skills/<name>/SKILL.md	Labs 9–11 (teammates load them)
Codex	.agents/skills/<name>/SKILL.md	Labs 8, 11
Hermes Agent	~/.hermes/profiles/<profile>/skills/	Labs 12–13

Tip: Do it once by hand, check it, then save it — never write a skill from a blank page.

The Four Studio Skills

What every agent on Day 2 needs to know about Horizon.

- **sunny-voice** — Plain English, banned words, the sign-off. Used for anything Horizon publishes.
- **fact-check** — Every number matched to facts-2026.md: VERIFIED or UNVERIFIED. Never fixed from memory.
- **fin-compliance** — Rules C1–C12, a table of issues, PASS or FAIL, and the right disclaimer.
- **channel-formats** — LinkedIn, Facebook, newsletter and YouTube shapes, UTM links and the content-file frontmatter.

Lab 7 — Studio Skills: Voice, Facts, Compliance and Formats

The story so far: Tomorrow a dozen agents will write for Horizon. Each one needs the same voice, the same facts and the same compliance rules — written once, used everywhere, and never pasted into a prompt again.

Goal: Tomorrow a dozen agents will write for Horizon. Write the voice, the facts, the compliance rules and the channel formats down once — as skills every agent loads when it needs them.

You'll build: Four skills — sunny-voice, fact-check, fin-compliance, channel-formats — in skills/, .claude/skills/ and .agents/skills/

Surface: Claude Cowork (/skill-creator) → every tool · **Time:** 40 min · **Slides:** 72–76

Lab folder: labs/lab-07-studio-skills/ — assets: skills-spec.md, channel-formats.md, brand.md, facts-2026.md, compliance-checklist.md

Step-by-step

1. **Do it once** — Paste Prompt A: Cowork writes one LinkedIn post by hand with you. Refine it until it is right.
2. **Save the skills** — Type /skill-creator and paste Prompt B.
3. **Write them to the project** — Cowork saves each skill folder in horizon-studio/skills/ as well.
4. **Share them** — Copy skills/* into .claude/skills/ (Claude Code) and .agents/skills/ (Codex).
5. **Test without naming them** — In a new Cowork task: "Write a Facebook post on SRS for young families." Ask which skills it used.

PROMPT A — Cowork: do it once

Write one LinkedIn post for Horizon on "Know your retirement number in 30 minutes", using data/brand.md, data/channel-formats.md and only facts in data/facts-2026.md. Then check it against data/compliance-checklist.md and show me a table of the checks. I will tell you what to change.

PROMPT B — /skill-creator: save it

/skill-creator Turn what we just did into four skills, as data/skills-spec.md describes: sunny-voice, fact-check, fin-compliance and channel-formats. Each has a "Use when..." description and stays under 80 lines. Also write each skill folder into horizon-studio/skills/<name>/SKILL.md so other tools can use the same files.

Check your work

- ☐ Four skills appear under Customize → Skills in Cowork.
- ☐ skills/ holds four folders, each with a SKILL.md.
- ☐ The same folders are in .claude/skills/ and .agents/skills/.
- ☐ Each description starts "Use when..."
- ☐ The new task picked the skills without being told to.
- ☐ fact-check marked a figure not on the facts sheet UNVERIFIED.

If it goes wrong

- **The skill never triggers** — Rewrite its description: say when to use it, with the words a person would type.

Stretch

- Add a fifth skill, utm-links, that tags every link with the right source, medium and campaign.

Why it matters: A skill costs almost nothing until it is needed. CLAUDE.md and AGENTS.md are read every time — keep them short; put procedures in skills.

Topic 2 recap

Day 1: Research, Build and Plan

Everything the content teams need tomorrow is ready.

1. **Researched** — Cited research and a recommendation (Lab 1).
2. **Live** — A checked website on GitHub Pages (Labs 2–3).
3. **Storyboarded** — One campaign story, three variants (Lab 4).
4. **Specified** — Personas, requirements, cadence, calendar (5–6).
5. **Equipped** — Four skills every agent can load (Lab 7).

Topic 3 — Multi-Channel Content Creation and Agent Workflow Coordination

Slides 78–111. In this topic you will:

- Subagents or an agent team?
- Human in the loop — enforced, not requested
- Social, newsletter and video teams

- From approval to LinkedIn, Facebook, email and YouTube

Key ideas for Lab 8

Subagents vs Agent Teams

Both run agents in parallel, each with its own context. The difference is who they talk to.

Subagents — a main agent starts them; each works in its own context and returns only its result: Researcher (returns facts); Writer (returns posts); Reviewer (returns a report).

Agent team — a team lead keeps a shared task list; teammates claim tasks and message each other directly: Researcher (sends facts); Writer (fixes on request); Reviewer (messages writer).

Use subagents when only the result matters. Use a team when the agents need to talk.

Which Pattern When?

From the Claude Code documentation, applied to the studio.

	Subagents	Agent team
Talks to	Returns a result to the caller	Teammates message each other directly
Coordination	The main agent manages everything	A shared task list; teammates claim tasks
Best for	Research, analysis, one-off reviews	Write ↔ review loops that need a conversation
Cost	Lower: results are summarised back	Higher: every teammate is its own Claude session
In the studio	Labs 1, 3, 4, 5, 14	Labs 9, 10, 11

Tip: Agent teams are experimental: turn them on with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`.

Anatomy of a Subagent

A Markdown file in `.claude/agents/`. The same file works as a subagent or as a teammate's agent type.

`.claude/agents/compliance-reviewer.md`

```

---
name: compliance-reviewer
description: Use before any Horizon content reaches a
  person. Checks facts and compliance, messages the
  writer, passes only when no Critical or High issue
  remains. Never edits.
tools: Read, Grep, Glob, SendMessage
model: opus
skills:
  - fact-check
  - fin-compliance
---
You are Horizon's compliance reviewer. You did not
write the content and you do not edit it. Send the
writer a table: words, rule, severity, fix.
Reply PASS only when no Critical or High issue
remains.

```

Running an Agent Team

Claude Code in a terminal. Describe the team in plain words.

1. **Turn it on** — "env": {"CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"} in .claude/settings.json; restart.
2. **Describe the team** — "Spawn three teammates using the agent types researcher, social-writer..."
3. **Watch** — ↑ ↓ and Enter open a teammate; Ctrl+T shows the task list.
4. **Steer** — Message any teammate directly; tell the lead to wait.
5. **Shut down** — "Ask the teammates to shut down" when the work is done.

Tip: One team per session. Teammates load CLAUDE.md, skills and MCP — but not the lead's conversation.

Human in the Loop, Enforced

A rule in a prompt persuades. Lab 8 builds a gate that holds.

- **Only people approve** — approve.mjs refuses to run without an interactive terminal — an agent's tool call cannot approve.
- **The approved words go out** — The approval stores a hash. Edit one word and publish --live refuses until a person approves again.
- **A hook stops agents** — A PreToolUse hook blocks agents from running approve.mjs, editing approvals.csv or publishing unapproved items.

The Hook That Holds the Gate

Claude Code runs it before every Bash, Edit or Write. Exit code 2 blocks the call and tells the agent why.

.claude/settings.json

```
{
  "env": { "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" },
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash|Edit|Write|MultiEdit",
      "hooks": [{
        "type": "command",
        "command": "node \"$CLAUDE_PROJECT_DIR/scripts/gate-hook.mjs\""}
      ]
    }]
  }
}
```

The Publishing APIs

publish.mjs is a dry run by default. Tokens live only in .env.

Channel	Endpoint	Permission
LinkedIn	POST api.linkedin.com/rest/posts (LinkedIn-Version: YYYYMM)	w_member_social — your own profile
Facebook	POST graph.facebook.com/<version>/<page-id>/feed	Page token: pages_manage_posts
YouTube	Resumable upload to youtube/v3/videos, privacy private	youtube.upload; unaudited projects stay private
Newsletter	No API — HTML + text files, then Gmail drafts	Consent = yes, unsubscribed = no

Tip: No accounts? Every lab works in dry-run mode: you see the exact request that would be sent.

Lab 8 — The Human Approval Gate and the Publishers

The story so far: Day 2. Agent teams are about to write for LinkedIn, Facebook and YouTube. Rachel's one condition: nothing goes out unless a person approved that exact version. Build the gate before the teams start.

Goal: Today agent teams will write for LinkedIn, Facebook and YouTube. Before any of them can post, build the gate: only a person can approve, and only the approved version can go out.

You'll build: scripts/submit.mjs, approve.mjs, publish.mjs, gate-hook.mjs, the hook in .claude/settings.json, and review/approvals.csv

Surface: Codex · **Time:** 35 min · **Slides:** 86–90

Lab folder: labs/lab-08-approval-gate-and-publishers/ — assets: publishing-spec.md, approvals-format.csv, connect-accounts.md, env.example

Step-by-step

1. **Add the spec** — Copy the assets into data/. Copy env.example to .env and add .env to .gitignore.
2. **Build the gate** — Paste Prompt A in Codex.
3. **Try to cheat** — Ask Codex to approve the test item itself. approve.mjs must refuse — it needs a person at a terminal.
4. **Approve it yourself** — In your own terminal: node scripts/approve.mjs test-001 --by "Your Name".

5. **Dry run** — Paste Prompt B. Read the exact request it would send.
6. **Connect accounts (optional)** — Follow connect-accounts.md to post for real later. Tokens go in .env only.

PROMPT A — Codex: build the gate

Build the publishing gate in data/publishing-spec.md, in Node 22 with no packages: scripts/submit.mjs, scripts/approve.mjs, scripts/publish.mjs and scripts/gate-hook.mjs, plus review/approvals.csv with the header in data/approvals-format.csv.

- approve.mjs refuses to run without an interactive terminal: only a person approves.
- publish.mjs is a dry run unless --live; live needs an approved row whose hash still matches, and never posts the same id twice.
- Add the PreToolUse hook to .claude/settings.json exactly as the spec says.
- Read tokens only from .env; never print one. Then create a test item content/social/test/test-001.md and submit it.

PROMPT B — Codex: dry run

Run node scripts/publish.mjs test-001 and show me the request it would send, with the token masked. Then edit one word in test-001.md and run it with --live. It must refuse because the content changed after approval. Explain the result in two lines.

Check your work

- ☐ Codex could not approve: approve.mjs refused without a terminal.
- ☐ You approved test-001 in your own terminal; the row shows your name, time and hash.
- ☐ The dry run printed the request with the token masked.
- ☐ After you edited the post, --live refused: hash changed.
- ☐ .claude/settings.json has the PreToolUse hook on Bash, Edit and Write.
- ☐ .env is git-ignored and holds no token you pasted into a chat.

If it goes wrong

- **Codex wants to run approve.mjs anyway** — That is the test. Say no — and confirm it refuses without a terminal.
- **LinkedIn returns 426** — LINKEDIN_VERSION must be a recent YYYYMM month that LinkedIn still supports.

Stretch

- Add a Slack or Telegram message to approve.mjs so the team sees every approval.

Why it matters: Rules in a prompt persuade; a gate enforces. The hook stops agents, approve.mjs needs a person, and the hash makes sure the approved words are the published words.

Key ideas for Lab 9

The Social Media Team

Lab 9: three teammates, five posts, one person deciding.

1. **Researcher** — Sends the writer a fact sheet per post.
2. **Social writer** — 2 LinkedIn + 3 Facebook posts, one file each.
3. **Compliance reviewer** — Messages the writer until each post passes.

4. **You** — Read every post; approve in your own terminal.
5. **Publisher** — Dry run, then --live when you say go.

Lab 9 — The Social Media Agent Team: LinkedIn and Facebook

The story so far: Week 1 of the campaign starts Monday: two LinkedIn posts, three Facebook posts. A researcher, a writer and a reviewer must agree on every word — and then a person decides.

Goal: Research, writing and review are different jobs. As a team, the researcher, the writer and the reviewer talk to each other directly and fix problems before a person ever sees a post.

You'll build: content/social/week-01/: 2 LinkedIn and 3 Facebook posts, reviewed, approved by you and posted (or dry-run)

Surface: Claude Code (subagents + agent teams) · **Time:** 45 min · **Slides:** 92–97

Lab folder: labs/lab-09-social-media-agent-team/ — assets: social-brief.md, team-roles.md, sample-calendar.csv

Step-by-step

1. **Turn on agent teams** — Paste Prompt A in Claude Code (claude in a terminal in horizon-studio), then restart it.
2. **Define the roles** — Claude writes three subagent files in .claude/agents/ from team-roles.md. Read one.
3. **Start the team** — Paste Prompt B. Three teammates appear in the panel under the prompt.
4. **Watch and steer** — Use ↑ ↓ and Enter to open a teammate; Ctrl+T shows the shared task list. Message the writer once.
5. **Review as a person** — Read each post in content/social/week-01/. Approve the good ones in your own terminal with approve.mjs.
6. **Publish** — Paste Prompt C. Dry run first; say "go" for --live only if your accounts are connected.
7. **Shut down** — Ask the lead to shut the teammates down.

PROMPT A — Claude Code: setup

Add "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" under "env" in .claude/settings.json without changing the hooks. Then create three project subagents in .claude/agents/ from data/team-roles.md: researcher, social-writer and compliance-reviewer — name, description, tools and model as the file says.

PROMPT B — Claude Code: the team

Create an agent team for Week 1 social content from data/social-brief.md (and strategy/calendar.csv if it exists). Spawn three teammates using the agent types researcher, social-writer and compliance-reviewer.

- researcher sends the writer a fact sheet per post, from data/facts-2026.md only.
- social-writer writes 2 LinkedIn and 3 Facebook posts, one file each in content/social/week-01/, with the frontmatter in data/publishing-spec.md.
- compliance-reviewer checks every post and messages the writer about each problem until it passes. A post is done only when the reviewer passes it; then run scripts/submit.mjs on it. Publish nothing.

PROMPT C — Claude Code: publish

For every Week 1 post that is approved in review/approvals.csv, run node scripts/publish.mjs <id> as a dry run and show me each request. Wait for me to say "go" before running any of them with --live.

Check your work

- ☐ Three subagent files exist, each with tools and a model.
- ☐ Three teammates appeared; the task list showed their tasks.
- ☐ The reviewer messaged the writer at least once, and the fix landed.
- ☐ Five posts with frontmatter and UTM links; all pending in approvals.csv.
- ☐ You approved in your own terminal; an agent attempt was blocked.
- ☐ Dry runs printed every request; live posts appear only if you said go.

If it goes wrong

- **You got subagents, not a team** — Check the env setting, restart claude, and ask explicitly for "an agent team".
- **Too many permission prompts** — Pre-approve Read, Write and node scripts/submit.mjs in .claude/settings.json.

Stretch

- Add a fourth teammate, community-manager, that drafts replies to the five most likely comments.

Why it matters: Subagents report to one boss; teammates also talk to each other. Use a team when the work needs a conversation — like a writer and a reviewer going back and forth.

Key ideas for Lab 10

The Newsletter Team

Lab 10: a task that waits for a person, then a list filtered by consent.

- **Write and review** — A researcher, a writer and the compliance reviewer agree on issue.md.
- **Wait for a person** — The builder's task depends on your approval. The lead keeps it blocked until you approve.
- **Consent decides** — Only consent = yes and unsubscribed = no: 31 of 40. Every email carries unsubscribe, address and disclaimer.

Lab 10 — The Newsletter Agent Team: the Sunny Sunday Email

The story so far: The Sunny Sunday email is the one channel that books chats reliably. November's issue needs the 2026 CPF figures exactly right — and must reach only the 31 people who opted in.

Goal: The newsletter turns checklist downloads into booked chats — but only if it is right, and only to people who said yes.

You'll build: content/newsletter/2026-11/: issue.md, newsletter.html, newsletter.txt, recipients.csv, excluded.csv, and Gmail drafts

Surface: Claude Code (agent team) → Claude Cowork (Gmail) · **Time:** 40 min · **Slides:** 99–103

Lab folder: labs/lab-10-newsletter-agent-team/ — assets: newsletter-brief.md, newsletter-spec.md, subscribers.csv

Step-by-step

1. **Start the team** — Paste Prompt A in Claude Code. Four teammates; the builder's task waits for your approval.
2. **Review the issue** — Read content/newsletter/2026-11/issue.md. Approve it in your own terminal: approve.mjs nl-2026-11.
3. **Release the builder** — Tell the lead it is approved. The builder makes the HTML, text and lists.
4. **Check the list** — recipients.csv must hold only consent = yes and unsubscribed = no.
5. **Create the drafts** — In Cowork with the Gmail connector, paste Prompt B.
6. **Send one test** — Send the draft addressed to yourself only, and read it on your phone.

PROMPT A — Claude Code: the team

Create an agent team for the November Sunny Sunday email from data/newsletter-brief.md and data/newsletter-spec.md. Spawn four teammates:

- researcher (agent type researcher): the facts.
- newsletter-writer: writes issue.md (id nl-2026-11) in our skills, then submit.mjs.
- compliance-reviewer (agent type compliance-reviewer): reviews until it passes.
- newsletter-builder: builds newsletter.html, newsletter.txt, recipients.csv and excluded.csv from data/subscribers.csv. The builder's task depends on a person approving nl-2026-11 in review/approvals.csv. It waits until I tell you it is approved and the hash matches.

PROMPT B — Cowork: Gmail drafts

Using the Gmail connector, create a DRAFT of the newsletter in content/newsletter/2026-11/newsletter.html for each address in recipients.csv, subject and preview from issue.md. Then one more draft addressed to me only. Do not send anything. Tell me how many drafts you made and how many people were excluded, and why.

Check your work

- ☐ Four teammates; the builder waited for your approval.
- ☐ The reviewer passed issue.md; you approved it in your terminal.
- ☐ newsletter.html has unsubscribe, address and disclaimer; newsletter.txt matches.
- ☐ recipients.csv holds 31 people; excluded.csv lists 9 with reasons.
- ☐ Cowork made drafts only; nothing was sent to the list.
- ☐ Your test email reads well on a phone.

If it goes wrong

- **The builder started early** — Say: "the build task is blocked until I approve — wait". Add a TaskCreated or TaskCompleted hook to enforce it.

Stretch

- Ask the builder for a second subject line to A/B test, with a reason for each.

Why it matters: Consent decides who gets mail — not the agent and not the deadline. Marketing to people who did not opt in breaks the PDPA.

Key ideas for Lab 11

The Video Team

Lab 11: a team writes the script; Codex makes the video.

- **Script as a team** — Scenes with seconds, words on screen and voice-over — reviewed like any post.
- **Render with Codex** — Branded title cards joined with ffmpeg; an optional voice-over. Approve the words AND the video.
- **Upload privately, disclose** — The upload is private. Say in the description that AI tools made the visuals; set YouTube's disclosure.

Lab 11 — The Video Agent Team: Script to YouTube

The story so far: Pre-retirees say they would rather watch than read. Rachel wants the first 'Money in Plain English' video: 60 seconds on the CPF retirement sums, uploaded privately until she has seen it.

Goal: Video reaches the people who never read posts. A team writes and checks the script; you approve it; Codex makes the video and uploads it — privately, until you decide to make it public.

You'll build: content/video/ep01/: scenes.json, ep01.mp4, ep01.md, and a private YouTube upload (or dry run)

Surface: Claude Code (agent team) → Codex (render, upload) · **Time:** 40 min · **Slides:** 105–110

Lab folder: labs/lab-11-video-agent-team-to-youtube/ — assets: youtube-brief.md, video-spec.md, sample-explainer.mp4

Step-by-step

1. **Script it as a team** — Paste Prompt A in Claude Code.
2. **Approve the script** — Read scenes.json and ep01.md, then approve yt-ep01-script in your own terminal.
3. **Render it** — In Codex, paste Prompt B. Watch the result.
4. **Approve the video** — Run submit.mjs on ep01.md, watch ep01.mp4 to the end, then approve yt-ep01.
5. **Upload** — Paste Prompt C. It goes up as private.
6. **Disclose** — In YouTube Studio, set the altered or synthetic content answer correctly before you make it public.

PROMPT A — Claude Code: the script team

Create an agent team for YouTube episode 1 from data/youtube-brief.md. Spawn three teammates: researcher (agent type researcher), a video-scriptwriter and compliance-reviewer (agent type compliance-reviewer). The scriptwriter writes content/video/ep01/ scenes.json (5-7 scenes: seconds, title, text) and ep01.md (id yt-ep01, channel youtube, title, tags, video content/video/ep01/ep01.mp4; the body is the description with chapters and the disclosure). Submit the script as yt-ep01-script once the reviewer passes it. Publish nothing.

PROMPT B — Codex: render

Following data/video-spec.md, write scripts/render-video.py and run it on the approved content/video/ep01/scenes.json to make content/video/ep01/ep01.mp4. Use ffmpeg and Pillow. Check the result with ffprobe: 1920x1080, under 90 seconds, every scene present. Do not change the approved words.

PROMPT C — Codex: upload

Run node scripts/publish.mjs yt-ep01 as a dry run and show me the request. If I say "go", run it with
--live: it uploads the video as PRIVATE. Give me the video id and the YouTube Studio link.

Check your work

- ☐ The reviewer passed the script; you approved yt-ep01-script.
- ☐ ep01.mp4 is 1920x1080, under 90 seconds, every scene present.
- ☐ Every figure on screen matches facts-2026.md.
- ☐ The description has chapters, UTM link, disclaimer and AI disclosure.
- ☐ You approved yt-ep01 after watching it to the end.
- ☐ The upload is private (or a dry run printed the request).

If it goes wrong

- **ffmpeg not found** — Install it (brew install ffmpeg / winget install ffmpeg) or use sample-explainer.mp4 to test the upload.
- **Upload forbidden** — Add yourself as a test user on the OAuth consent screen, then refresh the token.

Stretch

- Add a 30-second vertical Short from the same scenes.

Why it matters: Approve twice: once for the words, once for the finished video. Rendering can break what the script got right.

Topic 3 recap

Where You Are Now

Three agent teams produce reviewed content and a person approves every piece. But the studio still stops when you close the laptop.

- **Gate** · **Lab 8** — People approve, hashes hold, a hook blocks agents.
- **Teams** · **Labs 9–11** — Social, newsletter and video — research, write, review, approve.
- **What is missing** — A team that runs every week on any model, and a way to govern it. That is Topic 4.

Topic 4 — Content Distribution, Strategy Guidelines and Responsible AI Practices

Slides 112–141. In this topic you will:

- Hermes Agent: open-source agent bots on any model
- A Kanban board for a team of agents

- Always-on delivery: schedules and chat bots
- Measure, govern and write the playbook

Key ideas for Lab 12

What Is Hermes Agent?

Nous Research's open-source (MIT) agent. Your model, your machine, your rules.

The Model — Your model (Claude · GPT · open models): Any provider: Anthropic API, OpenRouter, Nous Portal or your own endpoint.

Hermes agent — open source:

- **Profiles** — Separate agents — own model, memory, skills and bot.
- **Kanban** — A shared board: tasks, parents, review, block.
- **Skills** — The same SKILL.md folders as Claude and Codex.
- **Gateway** — Telegram, Discord, Slack, WhatsApp, Signal, email.
- **Cron** — Jobs in plain words: "every Monday 8am".
- **Delegation** — `delegate_task` runs subagents in parallel.

A Claude subscription does not cover Hermes — use an API key with a spend limit.

The Hermes Kanban Board

Every task is a row anyone can read; every hand-off is visible.

1. **triage** — A rough idea; the planner breaks it down.
2. **todo** → **ready** — Ready when its parent tasks are done.
3. **running** — The dispatcher starts the assigned profile.
4. **blocked / review** — A person's turn: unblock or request changes.
5. **done** — Finished, with a summary for the next task.

Tip: The dispatcher runs inside hermes gateway start. hermes dashboard shows the board in a browser.

A Role Pipeline on the Board

Lab 12: researcher → writer → reviewer, linked by parents.

COMMANDS — terminal

```
hermes profile create researcher --description "Gathers facts."
hermes profile create writer --description "Writes in sunny-voice."
hermes profile create reviewer --description "Checks; blocks for a person."
hermes kanban init
hermes gateway start                                # runs the dispatcher

hermes kanban create "Facts for Week 2" --assignee researcher --json
hermes kanban create "Write Week 2" --assignee writer --parent <research-id>
hermes kanban create "Review Week 2" --assignee reviewer --parent <write-id>

hermes kanban watch                                # or: hermes dashboard
hermes kanban unblock <review-id>                   # your decision
```

Claude Agent Teams or Hermes Kanban?

Two ways to run a team. Choose by how long the work lives.

Claude Code agent team

- One session; the team ends with it
- Teammates message each other in real time
- Shared task list, kept while the session lasts
- Best for: a burst of collaborative work

Hermes Kanban

- A durable board shared by every profile
- Hand-offs through tasks, parents and comments
- Any model per profile; runs as a service
- Best for: weekly, always-on pipelines

Lab 12 — Hermes Agent Bots on a Kanban Board

The story so far: Rachel's IT adviser asks what happens if the studio's AI provider changes its prices or its rules. Try the same team on Hermes — open source, any model — with every hand-off on a Kanban board.

Goal: The studio should not depend on one vendor. Hermes runs a team of named agent bots on any model, coordinated on a durable Kanban board you can watch, block and unblock.

You'll build: Three Hermes profiles, a researcher → writer → reviewer pipeline on the board, and Week 2 content approved by you

Surface: Hermes Agent (profiles, Kanban) · **Time:** 45 min · **Slides:** 117–121

Lab folder: labs/lab-12-hermes-agent-bots-on-kanban/ — assets: hermes-setup.md, hermes-roles.md, kanban-pipeline.sh

Step-by-step

1. **Install Hermes** — Follow hermes-setup.md; hermes setup with the API key your trainer gives you.
2. **Create the bots** — Run the three hermes profile create commands in hermes-roles.md, then copy skills/ into each profile.
3. **Start the board** — hermes kanban init, then hermes gateway start in a second terminal — it runs the dispatcher.
4. **Build the pipeline** — Paste the prompt into hermes (or run kanban-pipeline.sh).
5. **Watch it** — hermes kanban watch, or hermes dashboard for the board in your browser.
6. **Be the gate** — The reviewer blocks for you. Read the output, then hermes kanban unblock <id> — or request changes.
7. **Hand it to the gate** — Save the approved post as content/social/week-02/li-w02-01.md and submit it.

PROMPT — Hermes: the planner

You are the planner for Horizon Wealth Planning. Create a Kanban pipeline for Week 2 content, with dependencies:

1. researcher: facts for a LinkedIn post and a newsletter tip on CPF cash top-ups, from

facts-2026.md only.

2. writer (parent: 1): write both in the sunny-voice skill.

3. reviewer (parent: 2): check with fact-check and fin-compliance; request changes until it passes, then block the task with the reason "Needs approval from a person". Work in ~/horizon-studio/hermes. Show me the task ids and the board.

COMMANDS — terminal

```
hermes profile create researcher --description "Gathers and checks facts for Horizon content."
hermes profile create writer --description "Writes Horizon posts in the sunny-voice skill."
hermes profile create reviewer --description "Checks Horizon content; blocks for human approval."
for p in researcher writer reviewer; do cp -R skills/* ~/.hermes/profiles/$p/skills/; done
hermes kanban init
hermes gateway start          # in a second terminal
hermes kanban watch           # or: hermes dashboard
hermes kanban unblock <task-id>
```

Check your work

- ☐ hermes profile list shows researcher, writer and reviewer.
- ☐ Each profile has the four studio skills.
- ☐ The board shows three tasks linked by parents.
- ☐ The writer started only after the researcher finished.
- ☐ The reviewer requested changes at least once, then blocked for you.
- ☐ You unblocked it; the post is submitted to approvals.csv.

If it goes wrong

- **Tasks stay in ready** — The gateway runs the dispatcher — start it with hermes gateway start.
- **Model errors** — A Claude subscription does not cover Hermes. Use the API key from your trainer.

Stretch

- Set a cheaper model for the researcher profile and a stronger one for the reviewer; compare the cost.

Why it matters: A Kanban board makes agent work visible and durable: every hand-off is a row anyone can read, and a blocked card is a person's turn.

Key ideas for Lab 13

Always-On Delivery

Lab 13: Hermes prepares, a person decides — from a phone.

- **A bot that answers only you** — hermes gateway setup connects Telegram. Allow only your own user id.
- **Schedules in plain words** — "Every Monday at 8am..." becomes a cron job. hermes cron list · run · pause.
- **Delivery targets** — telegram, slack, email or local files. The gateway must be running for jobs to fire.

Lab 13 — Always On: a Telegram Bot That Asks Before It Acts

The story so far: The studio must run every week, not just in class. Rachel lives on her phone. Have Hermes prepare the Monday brief on a schedule and send it to her — and wait for her answer.

Goal: A content studio runs every week, not just in class. Hermes prepares the Monday brief on a schedule and sends it to your phone — and waits for your answer.

You'll build: A Telegram bot, a Monday 8am content-brief job, and one approved brief from your phone

Surface: Hermes Agent (gateway bot, cron) · **Time:** 30 min · **Slides:** 123–126

Lab folder: labs/lab-13-always-on-telegram-bot/ — assets: telegram-bot-setup.md, cron-brief.md

Step-by-step

1. **Make the bot** — Follow telegram-bot-setup.md with @BotFather.
2. **Connect it** — hermes gateway setup → Telegram; allow only your own user id. Restart the gateway.
3. **Say hello** — Message the bot. Hermes answers.
4. **Schedule the brief** — Paste the prompt into Hermes.
5. **Run it now** — hermes cron list, then hermes cron run <id>.
6. **Approve from your phone** — Reply APPROVE, or ask for a change.
7. **Pause it** — After class: hermes cron pause <id>.

PROMPT — Hermes

Every Monday at 8am, prepare Horizon's weekly content brief as data/cron-brief.md describes, from ~/horizon-studio/strategy/calendar.csv, ~/horizon-studio/data/facts-2026.md and ~/horizon-studio/review/approvals.csv. Send it to me on Telegram and wait for my reply. Never publish, approve or send anything to anyone else.

Check your work

- ☐ The bot answers only you.
- ☐ hermes cron list shows the Monday 8am job.
- ☐ A manual run delivered the brief to Telegram.
- ☐ The brief lists the week's items, facts and pending approvals.
- ☐ Your reply was understood, and nothing was published.
- ☐ The job is paused after class.

If it goes wrong

- **No Telegram** — Use delivery "local" — the brief is saved as a file instead.
- **The job never runs** — Scheduled jobs run inside the gateway — keep hermes gateway running.

Stretch

- Add a Friday 5pm job that summarises what was published and what is still pending.

Why it matters: Automate the preparation, not the decision. The schedule fills your inbox; you still press the button.

Key ideas for Lab 14

Responsible AI in the Studio

Each risk has a control you built in this course.

Risk	Control	Where
Wrong figure	fact-check skill; reviewer agents; facts sheet	Labs 3, 7, 9–12
Read as advice or a promise	fin-compliance; disclaimers; no returns	Labs 3, 7
Unapproved post	approve.mjs (people only), hash, hook	Lab 8
Marketing without consent	recipients filtered on consent and unsubscribe	Lab 10
Undisclosed AI media	Description line + YouTube disclosure	Lab 11
Runaway automation	Bot answers only you; jobs paused	Lab 13

Tip: Keep keys in .env, client data out of prompts, and the approval records for audit.

Measure What Matters

Lab 14: the numbers decide next month — and which agents earn their tokens.

- **Cost per chat** — Clicks are vanity. Spend ÷ chats booked shows which channel works.
- **Defects before a person** — A team that catches mistakes before review saves the reviewer's time.
- **The playbook** — Roster, workflow, approval matrix, rules, incidents, review — so the studio runs without you.

Lab 14 — Measure, Govern and Write the Playbook

The story so far: Eight weeks in. Rachel asks two questions: which channels actually booked chats, and how does the team run this safely after you leave? Answer with numbers, then with a playbook.

Goal: Rachel asks two questions: what worked, and how do we run this safely every week? Answer the first with the numbers and the second with a playbook anyone on the team can follow.

You'll build: A results dashboard (Live Artefact), reports/agent-comparison.md and strategy/content-strategy-playbook.md

Surface: Claude Cowork (sub-agents, Live Artefact) · **Time:** 40 min · **Slides:** 129–133

Lab folder: labs/lab-14-measure-govern-playbook/ — assets: campaign-results.csv, run-log.csv, playbook-outline.md, responsible-ai-checklist.md

Step-by-step

1. **Add the data** — Copy the assets into data/.
2. **Analyse in parallel** — Paste Prompt A — two sub-agents.
3. **Read the dashboard** — Which channel books a chat most cheaply? Where should next month's S\$4,000 go?
4. **Write the playbook** — Paste Prompt B.

5. **Review it as Rachel** — Ask the compliance reviewer to check it, then add one rule of your own.

PROMPT A — Cowork: the analysts

Use two sub-agents in parallel.

1. performance-analyst: from data/campaign-results.csv, per channel — clicks, checklist downloads, chats booked and cost per chat. Which channel books chats most cheaply, and how should next month's S\$4,000 be split? Build a Live Artefact dashboard.
2. agent-analyst: from data/run-log.csv, compare Cowork sub-agents, Codex subagents, Claude Code agent teams and Hermes Kanban on minutes, tokens, defects caught before a person, and human edits. Save reports/agent-comparison.md.

PROMPT B — Cowork: the playbook

Write strategy/content-strategy-playbook.md with data/playbook-outline.md, using everything in research/, strategy/, review/ and reports/. Include the agent roster (job, tool, model, what it may never do), the approval matrix by risk, the rules in data/responsible-ai-checklist.md, and what to do when a wrong post goes out. Keep it to what a new team member can follow on their first day.

Check your work

- ☐ The dashboard shows cost per chat for every channel.
- ☐ The budget split follows the numbers, with reasons.
- ☐ agent-comparison.md shows the baseline caught 0 defects before a person.
- ☐ The playbook names every agent and bot, with its limits.
- ☐ The approval matrix says who approves CPF and tax figures.
- ☐ It has an incident procedure, and your own rule is in it.

If it goes wrong

- **The playbook is generic** — Say: "use our actual agents, files and numbers — no generic advice".

Stretch

- Turn the playbook into a skill, studio-playbook, so every agent can check its own work against it.

Why it matters: Teams cost more tokens. The run log shows what you get for them: defects caught before a person, and fewer human edits.

Course Summary

How Each Pattern Is Triggered

Five ways to run many agents. The difference is what sets each one off.

- **Sub-agents (by delegation)** — The main agent starts them when you ask — or when a task matches a subagent's description. Example: "use sub-agents, one per role". Where: Cowork · Codex · .claude/agents/.
- **Agent teams (by the lead)** — The lead spawns teammates who share a task list and message each other. Example: "spawn three teammates...". Where: Claude Code (experimental).

- **Skills (on demand)** — Loaded when the task matches the description. Free until then.
Example: fact-check fin-compliance. Where: Every tool, same SKILL.md.
- **Hooks (on an event)** — Run every time an event fires — before a tool runs. The model cannot skip them. Example: PreToolUse: gate-hook.mjs. Where: .claude/settings.json.
- **Kanban + cron (on the board / clock)** — A task runs when its parents finish; a job runs on its schedule. Example: every monday 8am --parent <id>. Where: Hermes gateway.

Who decides? You: the approval · the model: delegation and skills · the system, every time: hooks, the board and the clock

Horizon: Research to an Always-On Studio

Two days, four tools, one regulated business.

1. **Research** — Cowork sub-agents: a cited recommendation.
2. **Build** — Codex: a checked website, live on Pages.
3. **Plan** — Storyboard, personas, cadence and shared skills.
4. **Create** — Agent teams for social, email and video, gated.
5. **Run** — Hermes bots on Kanban, measured and governed.

Quick Command Reference

Command	What it does
Cowork: "use sub-agents, one per role"	Run specialists in parallel, each in its own context
Cowork: /skill-creator	Save what you just did as a skill
Codex: /plan · /init	Plan before editing · draft AGENTS.md
Codex: "use a subagent as..."	Delegate a focused job, e.g. a reviewer that only reports
claude	Start Claude Code in the current folder
.claude/agents/<name>.md	A subagent definition: name, description, tools, model, skills
"CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"	Under "env" in .claude/settings.json — turns on agent teams
↑ ↓ · Enter · Ctrl+T	Agent team: select a teammate · open it · show the task list
node scripts/submit.mjs <file>	Agents: put a content file up for review (pending)
node scripts/approve.mjs <id> --by "Name"	People only, in your own terminal: approve this version
node scripts/publish.mjs <id> [--live]	Dry run by default; --live posts an approved, unchanged item
python3 scripts/render-video.py <scenes.json>	Render the approved script into an MP4
hermes setup · hermes model	Configure Hermes Agent and its model
hermes profile create <name> --description "..."	A new Hermes agent (bot) with its own model and skills
hermes kanban init · create · watch · unblock	Run a team of profiles on the Kanban board
hermes gateway setup · start	Connect Telegram; run the dispatcher and scheduled jobs
hermes cron list · run · pause <id>	Manage scheduled jobs
python3 -m http.server 8080 --directory	Serve the website locally

Support

Tertiary Infotech Academy Pte Ltd · enquiry@tertiaryinfotech.com · +65 6100 0613 · www.tertiarycourses.com.sg

Courseware and the assessment are on the LMS: <https://lms-tms.tertiaryinfotech.com/>

Assessment flow

1. TRAQOM — scan the TRAQOM QR code on the LMS and complete the survey.
2. Assessment Digital Attendance.
3. Assessment — Written Assessment (1 hour, from 4:00 PM) and Practical Performance (1 hour, from 5:00 PM).
4. Submit the assessment answers on the LMS.
5. Sign the Assessment Summary Record.